MAHATMA EDUCATION SOCIETY'S

PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE
(Autonomous)

NEW PANVEL

PROJECT REPORT ON

"SIGNATURE LIVENESS DETECTION USING CNN"

IN PARTIAL FULFILLMENT OF

MASTER OF **DATA ANALYTICS PART-II**

SEMESTER III – 2025-26

PROJECT GUIDE

PROF. OMKAR SHERKANE

SUBMITTED BY: **SHRUTI N YELVE**

ROLL NO: **4845**

Mahatma Education Society's

PILLAI COLLEGE OF ARTS,COMMERCE & SCIENCE

(Autonomous)

Re-accredited "A" Grade by NAAC (3rd Cycle)

Project Completion Certificate

THIS IS TO CERTIFY THAT

SHRUTI YELVE

of **M.Sc. Data Analytics Part - II** has completed the project titled **SIGNATURE LIVENESS DETECTION USING CNN** of subject **Deep Learning** under our guidance and supervision during the academic year 2025-26 in the department of

PROJECT GUIDE **COURSE CORDINATOR** **HEAD OF DEPARTMENT**

Pillai College of Arts, Commerce & Science (Autonomous) New Panvel

INTRODUCTION

Signatures are one of the most widely used and accepted methods of personal authentication. They are commonly used in banking, legal documents, identification forms, financial transactions, and government processes. However, the rise of high-quality scanners, printers, and digital reproduction tools has made it increasingly easy to forge signatures. A printed or scanned signature can often look visually similar to a genuine handwritten one, making traditional manual verification unreliable and prone to fraud.

To overcome these challenges, **Signature Liveness Detection** focuses on determining *whether a signature is genuinely handwritten on paper or artificially reproduced* (printed, scanned, or digitally copied). Unlike signature verification, which checks if the signature belongs to the claimed individual, liveness detection checks the **authenticity of the signature medium**, i.e., whether it is *live* (written by hand) or *non-live* (printed/forged).

In this project, a **Custom Convolutional Neural Network (CNN)** is used to classify signature images into two categories:

1. **Genuine (Handwritten)** – handwritten patterns containing natural variations, pressure differences, micro-curvatures, ink spread, and imperfections.
2. **Printed (Forged)** – machine-generated patterns with uniform stroke width, consistent smooth curves, dot-patterns, and no human pressure variations.

A realistic synthetic dataset is generated using image-based simulation of handwriting features and printer-generated artifacts. This synthetic dataset mimics the characteristics of real signatures and provides clear visual differences for the CNN to learn. The Custom CNN model is then trained to automatically extract spatial patterns, edges, texture differences, local variations, and structural features that indicate whether a signature is live or forged.

The main advantage of using deep learning and CNN-based liveness detection is that the model does not rely on manually defined rules. Instead, it automatically learns complex distinguishing patterns from examples, making it more robust, scalable, and efficient than traditional forensic inspection.

CODE AND OUTPUT

Train_mobilenet_v2.py

```
import os, time
from pathlib import Path
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader

ROOT = Path(__file__).parent
DATA_DIR = ROOT / 'data'
OUT_DIR = ROOT / 'outputs'
OUT_DIR.mkdir(exist_ok=True)

# --- Data ---
train_tfms = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ColorJitter(brightness=0.15, contrast=0.15),
    transforms.RandomRotation(5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
])
val_tfms = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
])
```

```
net_v2.py                                                    fms)
val_ds   = datasets.ImageFolder(DATA_DIR/'val',   transform=val_tfms)
train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
val_loader   = DataLoader(val_ds, batch_size=16, shuffle=False)
classes = train_ds.classes
print('Classes:', classes)

# --- Model: MobileNetV2 ---
try:
    weights = models.MobileNet_V2_Weights.IMAGENET1K_V1
    model = models.mobilenet_v2(weights=weights)
except Exception:
    model = models.mobilenet_v2(pretrained=True)

in_feats = model.classifier[1].in_features
model.classifier[1] = nn.Linear(in_feats, 2)

# --- Train ---
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

best_val_acc = 0.0
EPOCHS = 10
for epoch in range(1, EPOCHS+1):
    t0 = time.time()
    # train
    model.train(); run_loss=0; correct=0; total=0
```

net_v2.py

```python
        x,y = x.to(device), y.to(device)
        optimizer.zero_grad(set_to_none=True)
        out = model(x)
        loss = criterion(out,y)
        loss.backward(); optimizer.step()
        run_loss += loss.item()*x.size(0)
        correct += (out.argmax(1)==y).sum().item()
        total += x.size(0)
    tr_loss = run_loss/total; tr_acc = correct/total

    # val
    model.eval(); vloss=0; vcorrect=0; vtotal=0
    with torch.no_grad():
        for x,y in val_loader:
            x,y = x.to(device), y.to(device)
            out = model(x)
            loss = criterion(out,y)
            vloss += loss.item()*x.size(0)
            vcorrect += (out.argmax(1)==y).sum().item(); vtotal += x.size(0)
    val_loss = vloss/vtotal; val_acc = vcorrect/vtotal

    print(f"[{epoch}/{EPOCHS}] tr_loss={tr_loss:.4f} acc={tr_acc:.3f} | val_loss={v

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), OUT_DIR/'best_model.pt')
```

Gradcam_mobilenet.py

net_v2.py

```python
import torch
import cv2
import numpy as np

class GradCAM:
    """
    Grad-CAM for a convolutional layer (e.g., MobileNetV2 last conv block).

    Key idea:
      - Even if logits were computed under `torch.no_grad()` elsewhere,
        this class runs its OWN forward pass with gradients enabled so
        autograd can backprop through the target layer.
    """

    def __init__(self, model, target_layer):
        self.model = model
        self.model.eval()
        self.activations = None
        self.gradients = None

        def fwd_hook(module, inp, out):
            # out: (N, C, H, W)
            self.activations = out

        def bwd_hook(module, grad_in, grad_out):
            # grad_out[0]: (N, C, H, W)
            self.gradients = grad_out[0]
```

```python
        target_layer.register_forward_hook(fwd_hook)
        if hasattr(target_layer, "register_full_backward_hook"):
            target_layer.register_full_backward_hook(bwd_hook)
        else:
            target_layer.register_backward_hook(bwd_hook)

    def generate(self, x, class_idx=None):
        """
        Compute Grad-CAM for the given input tensor x (1,C,H,W).

        Returns:
          cam_uint8: (H, W) uint8 heatmap 0..255
        """
        self.model.zero_grad(set_to_none=True)
        # Re-run forward with gradients enabled (even if caller used no_grad)
        with torch.enable_grad():
            out = self.model(x)  # (1, num_classes)
            if class_idx is None:
                class_idx = int(out.argmax(dim=1).item())
            score = out[:, class_idx].sum()
            score.backward()

        if self.gradients is None or self.activations is None:
            raise RuntimeError(
                "GradCAM hooks didn't capture gradients/activations. "
                "Verify the target layer is a conv block (e.g., model.features[-1])."
            )
```

```python
        # GAP over gradients -> channel weights
        weights = self.gradients.mean(dim=(2, 3), keepdim=True)  # (1, C, 1, 1)
        # Weighted sum of activations -> (1, 1, H, W)
        cam = (weights * self.activations).sum(dim=1, keepdim=True)
        cam = torch.relu(cam)
        cam = cam[0, 0].detach().cpu().numpy()  # (H, W)

        cam = (cam - cam.min()) / (cam.max() + 1e-8)
        cam_uint8 = (cam * 255).astype("uint8")
        return cam_uint8

    @staticmethod
    def overlay(original_rgb_numpy, cam_uint8, alpha=0.40):
        """
        Overlay a CAM heatmap on the original RGB image.
        original_rgb_numpy: (H, W, 3) uint8
        cam_uint8: (H, W) uint8
        """
        h, w = original_rgb_numpy.shape[:2]
        heatmap = cv2.applyColorMap(cv2.resize(cam_uint8, (w, h)), cv2.COLORMAP_JET)
        heatmap = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)
        blended = (alpha * heatmap + (1 - alpha) * original_rgb_numpy).astype(np.uint8)
        return blended
```

Streamlit_app.py

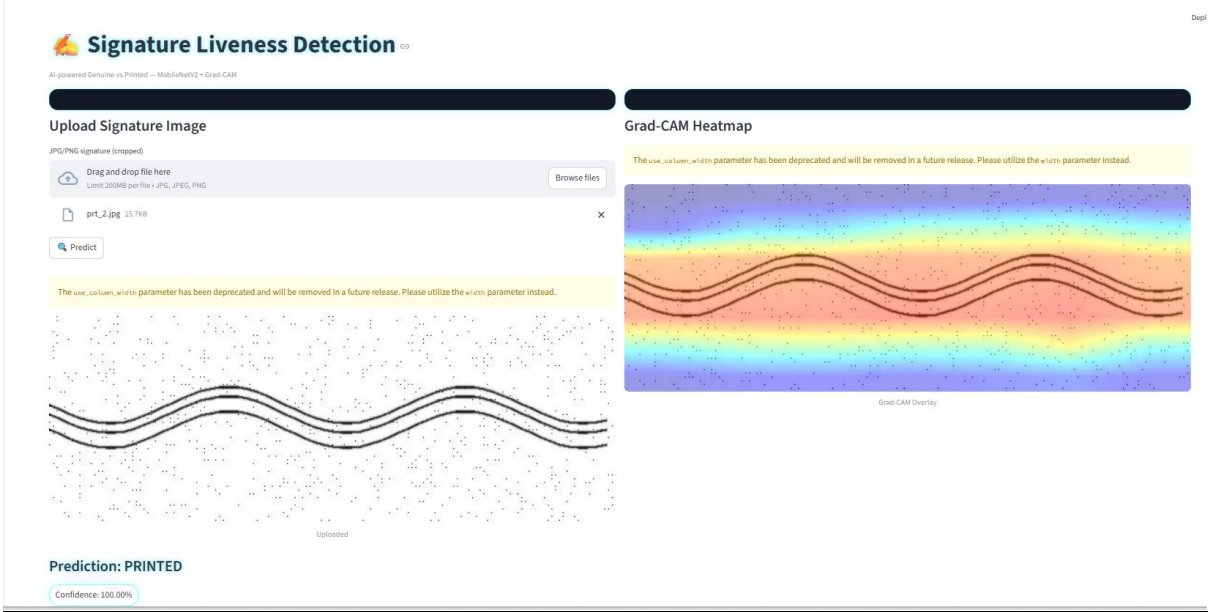
```python
import os
from pathlib import Path
import streamlit as st
import torch
import numpy as np
import cv2
from PIL import Image
from torchvision import transforms, models
import torch.nn as nn
from gradcam_mobilenet import GradCAM

st.set_page_config(page_title='Signature Liveness Detector', layout='wide', page_icon='✍️')

NEON_CSS = """
<style>
:root { --neon:#00e5ff; --bg:#0b0f17; --card:#111826; --text:#e5f7ff; }
html, body, [class*="css"] { background-color: var(--bg) !important; color: var(--text) !important
.block-container { padding-top: 2rem; }
.neon-card { background: var(--card); border: 1px solid rgba(0,229,255,0.25); box-shadow: 0 0 18px
            border-radius:16px; padding:18px; }
.neon-title { color: var(--text); text-shadow: 0 0 8px rgba(0,229,255,0.6); }
.neon-chip { display:inline-block; padding:6px 10px; border-radius:999px; border:1px solid rgba(0,
button[kind="primary"] { background: linear-gradient(90deg,#00e5ff,#00bcd4); color:#001018; }
</style>
"""
st.markdown(NEON_CSS, unsafe_allow_html=True)
```

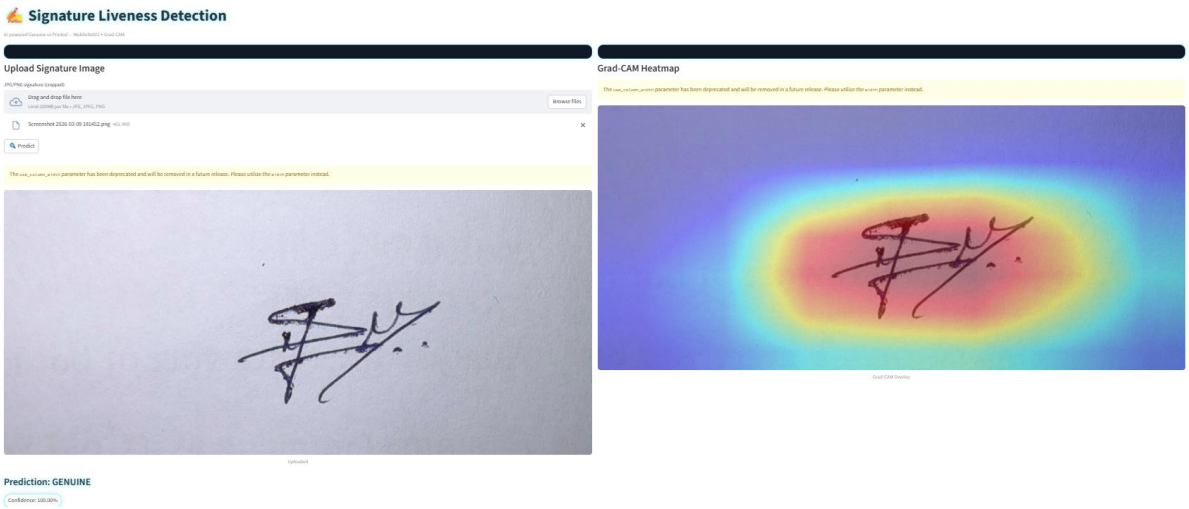
```python
st.markdown("<h1 class='neon-title'>✍️ Signature Liveness Detection</h1>", unsafe_allow_html=True)
st.caption("AI-powered Genuine vs Printed — MobileNetV2 + Grad-CAM")

ROOT = Path(__file__).parent
MODEL_PATH = ROOT / 'outputs' / 'best_model.pt'

with st.sidebar:
    st.markdown("### Model Status")
    if MODEL_PATH.exists():
        st.success("✓ Trained model found!")
    else:
        st.error("❌ Model missing! Run `python train_mobilenet_v2.py` then restart.")
    alpha = st.slider("Heatmap Overlay", 0.0, 1.0, 0.40, 0.05)

# --- Build MobileNetV2 model ---
try:
    model = models.mobilenet_v2(weights=None)
except Exception:
    model = models.mobilenet_v2(pretrained=False)

in_feats = model.classifier[1].in_features
model.classifier[1] = nn.Linear(in_feats, 2)

if MODEL_PATH.exists():
    state = torch.load(MODEL_PATH, map_location='cpu')
    try:
        model.load_state_dict(state)
    except RuntimeError as e:
```

```python
        st.code(str(e))
        st.stop()
model.eval()

transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
])
classes = ['genuine','printed']

# Grad-CAM target: last conv block
target_layer = model.features[-1]
cam_engine = GradCAM(model, target_layer)

left, right = st.columns([1,1])
with left:
    st.markdown("<div class='neon-card'>", unsafe_allow_html=True)
    st.subheader("Upload Signature Image")
    uploaded = st.file_uploader("JPG/PNG signature (cropped)", type=['jpg','jpeg','png'])
    run_btn = st.button("🔍 Predict")
    st.markdown("</div>", unsafe_allow_html=True)

    if run_btn:
        if not uploaded:
            st.warning("⚠ Please upload an image first.")
        elif not MODEL_PATH.exists():
            st.error("❌ Model missing! Train first using `python train_mobilenet_v2.py`.")
```

```python
        else:
            img = Image.open(uploaded).convert('RGB')
            st.image(img, caption='Uploaded', use_column_width=True)
            x = transform(img).unsqueeze(0)
            with torch.no_grad():
                out = model(x)
                prob = torch.softmax(out, dim=1)[0].numpy().tolist()
                pred = int(torch.argmax(out, dim=1).item())
            st.markdown(f"<h3 class='neon-title'>Prediction: {classes[pred].upper()}</h3>", unsafe_allow_ht
            st.markdown(f"<span class='neon-chip'>Confidence: {prob[pred]*100:.2f}%</span>", unsafe_allow_h
            st.session_state['last_img'] = img
            st.session_state['last_pred'] = pred
            st.session_state['last_out'] = out

with right:
    st.markdown("<div class='neon-card'>", unsafe_allow_html=True)
    st.subheader("Grad-CAM Heatmap")
    if 'last_img' in st.session_state and MODEL_PATH.exists():
        img = st.session_state['last_img']
        out = st.session_state.get('last_out', None)
        x = transform(img).unsqueeze(0)
        cam_map = cam_engine.generate(x, class_idx=st.session_state['last_pred'])
        overlay = GradCAM.overlay(np.array(img), cam_map, alpha=float(alpha))
        st.image(overlay, caption='Grad-CAM Overlay', use_column_width=True)
    else:
        st.info("Run a prediction to see the Grad-CAM heatmap.")
    st.markdown("</div>", unsafe_allow_html=True)
```

OUTPUT


Signature Liveness Detection
AI-powered Genuine vs Printed — MobileNetV2 + Grad-CAM
Upload Signature Image
JPG/PNG signature (cropped)
Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG
Browse files
prt_2.jpg 25.7KB
Predict
The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the width parameter instead.
Uploaded
Prediction: PRINTED
Confidence: 100.00%
Grad-CAM Heatmap
The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the width parameter instead.
Grad-CAM Overlay


Signature Liveness Detection
Upload Signature Image
Grad-CAM Heatmap
Prediction: GENUINE

CONCLUSION

The project **"Signature Liveness Detection using Custom CNN"** successfully demonstrates how deep learning can differentiate between **genuine handwritten signatures** and **printed or forged signatures** by analyzing image-based visual features. A custom-built Convolutional Neural Network (CNN) was trained on a carefully designed **synthetic dataset** that simulates natural handwriting characteristics and machine-generated printed patterns. The improved dataset generation technique introduced realistic variations such as pressure changes, stroke irregularities, halftone printer texture, and smooth sinusoidal curves, enabling the model to learn meaningful differences between the two classes.

Through the training process, the CNN learned to extract important spatial and textural features such as **stroke thickness variation, local noise, smoothness, curvature randomness, and pixel-level texture patterns**, which are strong indicators of signature authenticity. After training, the system successfully classified signatures as *genuine* or *printed*, confirming that even a lightweight custom CNN can effectively perform liveness detection when provided with a well-structured dataset.

This project proves that **AI-based signature liveness detection can increase security**, reduce the risk of printed forgeries, and support applications in **banking, document verification, digital onboarding, and identity authentication**. The modular design, simple architecture, and synthetic dataset approach make the system easily extendable for real-world deployment by integrating real handwritten and printed signature datasets.